

AAI590 Capstone — Group 4

Module 3 | Notebook 2: Exploratory Data Analysis (EDA)

Project: ML-Based Murmur Detection for RHD Screening Using Phonocardiogram Data

Dataset: CirCor DigiScope Phonocardiogram Dataset v1.0.3 (PhysioNet)

Authors: Shiva Bathula · Kaushik M · Pavan Kumar Bellary

Notebook Goals

1. Load verified file index from Notebook 1
2. Class balance analysis — Murmur and Outcome labels
3. Demographic analysis — Age, Sex distributions
4. Recording duration distribution across sites and labels
5. Signal quality audit — energy-based screening
6. Per-site class balance analysis
7. Sample waveform comparison — Normal vs Abnormal
8. MFCC feature heatmap comparison
9. Inter-variable correlation analysis
10. Summary of findings and modelling implications

Session Recovery from Previous Notebook

```
In [2]: # — Cell 0: Session Recovery — run this first if files are missing —
import os, subprocess, getpass
from pathlib import Path

print('Checking for required files...')

files_needed = {
    '/content/circor_file_index.csv': False,
    '/content/circor-heart-sound/training_data.csv': False,
}

for f in files_needed:
    files_needed[f] = Path(f).exists()
    status = ' ' if files_needed[f] else ' '
    print(f'{status} {f}')

if not all(files_needed.values()):
    print('\nFiles missing — downloading from GitHub repo...')

    GITHUB_USER = input('GitHub username: ')
    GITHUB_TOKEN = getpass.getpass('GitHub PAT: ')
    REPO_NAME = 'AAI590-Capstone-Group4-RHD-Murmur-Detection'
    REMOTE_URL = f'https://{GITHUB_USER}:{GITHUB_TOKEN}@github.com/{GIT

    # Clone repo to get the CSV files back
```

```

os.system('rm -rf /content/repo')
os.system(f'git clone {REMOTE_URL} /content/repo')

# Restore file index from repo
if Path('/content/repo/data/circor_file_index.csv').exists():
    import shutil
    shutil.copy('/content/repo/data/circor_file_index.csv',
                '/content/circor_file_index.csv')
    print('  circor_file_index.csv restored from GitHub')

# Re-download WAV files and training_data.csv from PhysioNet
print('\nRe-downloading dataset from PhysioNet...')
PHYSIONET_USER = input('PhysioNet username: ')
PHYSIONET_PASS = getpass.getpass('PhysioNet password: ')

os.makedirs('/content/circor-heart-sound', exist_ok=True)
os.system(f'wget -r -N -c -np '
          f'--user="{PHYSIONET_USER}" '
          f'--password="{PHYSIONET_PASS}" '
          f'-P /content/circor-heart-sound '
          f'https://physionet.org/files/circor-heart-sound/1.0.3/trai
os.system(f'wget -N '
          f'--user="{PHYSIONET_USER}" '
          f'--password="{PHYSIONET_PASS}" '
          f'-P /content/circor-heart-sound '
          f'https://physionet.org/files/circor-heart-sound/1.0.3/trai

# Flatten nested directory
import shutil
nested = Path('/content/circor-heart-sound/physionet.org/files/circor
DATA_ROOT = Path('/content/circor-heart-sound')
if nested.exists():
    for item in nested.iterdir():
        dest = DATA_ROOT / item.name
        if not dest.exists():
            shutil.move(str(item), str(DATA_ROOT))
    shutil.rmtree('/content/circor-heart-sound/physionet.org')

# Final check
print('\n— Final file check —')
for f in ['/content/circor_file_index.csv',
          '/content/circor-heart-sound/training_data.csv',
          '/content/circor-heart-sound/training_data']:
    exists = Path(f).exists()
    print(f'{f} " if exists else "" } {f}')
else:
    print('\n  All files present – proceed to Cell 1.')

```

```
Checking for required files...
[] /content/circor_file_index.csv
[] /content/circor-heart-sound/training_data.csv

Files missing – downloading from GitHub repo...
GitHub username: ShivaNagaVaraPrasadBathula
GitHub PAT: .....
[] circor_file_index.csv restored from GitHub

Re-downloading dataset from PhysioNet...
PhysioNet username: shivab
PhysioNet password: .....

— Final file check —
[] /content/circor_file_index.csv
[] /content/circor-heart-sound/training_data.csv
[] /content/circor-heart-sound/training_data
```

1. Setup & Load Data

```
In [3]: # — Install & import —
!pip install librosa soundfile --quiet

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
import librosa
import librosa.display
import warnings
warnings.filterwarnings('ignore')
from pathlib import Path
from collections import Counter

# — Plot style —
plt.rcParams.update({
    'figure.dpi': 120,
    'axes.spines.top': False,
    'axes.spines.right': False,
    'font.family': 'DejaVu Sans'
})
NAVY = '#1F4E79'
BLUE = '#2E75B6'
STEEL = '#5BA3D0'
GOLD = '#C9A84C'
GREEN = '#4F7942'
RED = '#C0392B'
COLORS = [BLUE, GOLD, GREEN, RED, STEEL]

# — Paths —
DATA_ROOT = Path('/content/circor-heart-sound')
AUDIO_DIR = DATA_ROOT / 'training_data'
LABEL_CSV = DATA_ROOT / 'training_data.csv'
INDEX_CSV = Path('/content/circor_file_index.csv')
OUTPUT_DIR = Path('/content/eda_outputs')
```

```

OUTPUT_DIR.mkdir(exist_ok=True)

# — Load file index and label CSV —————
file_index = pd.read_csv(INDEX_CSV)
df          = pd.read_csv(LABEL_CSV)

print(f'File index shape : {file_index.shape}')
print(f'Label CSV shape  : {df.shape}')
print(f'Columns          : {df.columns.tolist()}')

```

```

File index shape : (3163, 5)
Label CSV shape  : (942, 23)
Columns          : ['Patient ID', 'Recording locations:', 'Age', 'Sex', 'Height', 'Weight', 'Pregnancy status', 'Murmur', 'Murmur locations', 'Most audible location', 'Systolic murmur timing', 'Systolic murmur shape', 'Systolic murmur grading', 'Systolic murmur pitch', 'Systolic murmur quality', 'Diastolic murmur timing', 'Diastolic murmur shape', 'Diastolic murmur grading', 'Diastolic murmur pitch', 'Diastolic murmur quality', 'Outcome', 'Campaign', 'Additional ID']

```

2. Class Balance Analysis

```

In [4]: # — Murmur and Outcome label distributions —————
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Murmur label
murmur_counts = df['Murmur'].value_counts()
bars = axes[0].bar(murmur_counts.index, murmur_counts.values,
                  color=[BLUE, GOLD, GREEN], edgecolor='white', linewidth=1)
axes[0].set_title('Murmur Label Distribution', fontweight='bold', fontsize=13)
axes[0].set_xlabel('Murmur Label')
axes[0].set_ylabel('Patient Count')
for bar, val in zip(bars, murmur_counts.values):
    pct = val / len(df) * 100
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 5,
                 f'{val}\n({pct:.1f}%)', ha='center', fontsize=10)

# Outcome label
outcome_counts = df['Outcome'].value_counts()
bars2 = axes[1].bar(outcome_counts.index, outcome_counts.values,
                   color=[BLUE, GOLD], edgecolor='white', linewidth=1.5)
axes[1].set_title('Clinical Outcome Distribution', fontweight='bold', fontsize=13)
axes[1].set_xlabel('Outcome Label')
axes[1].set_ylabel('Patient Count')
for bar, val in zip(bars2, outcome_counts.values):
    pct = val / len(df) * 100
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 5,
                 f'{val}\n({pct:.1f}%)', ha='center', fontsize=10)

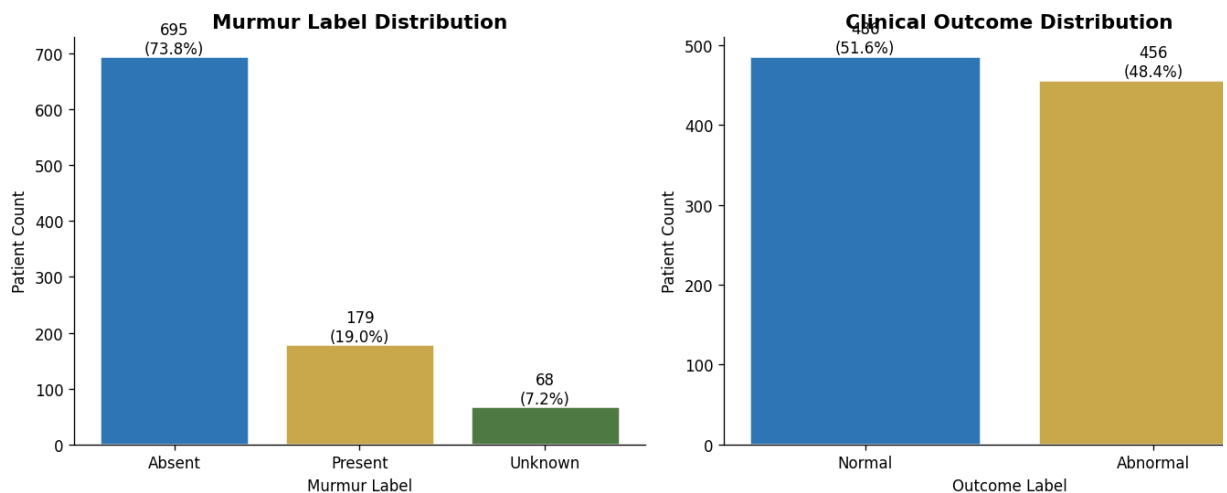
plt.suptitle('CirCor DigiScope — Target Variable Distributions (n=942 patients)',
             fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'class_balance.png', bbox_inches='tight')
plt.show()

# Key findings
print('— Class balance summary —————')
print(f'Murmur: Absent={murmur_counts.get("Absent",0)}, '

```

```
f'Present={murmur_counts.get("Present",0)}, '
f'Unknown={murmur_counts.get("Unknown",0)}')
print(f'Outcome: Normal={outcome_counts.get("Normal",0)}, '
      f'Abnormal={outcome_counts.get("Abnormal",0)}')
print(f'Outcome imbalance ratio: {outcome_counts.max()/outcome_counts.min()})
```

CirCor DigiScope — Target Variable Distributions (n=942 patients)



— Class balance summary —

Murmur: Absent=695, Present=179, Unknown=68

Outcome: Normal=486, Abnormal=456

Outcome imbalance ratio: 1.07:1

3. Demographic Analysis

```
In [5]: # — Age group and Sex distributions —
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Age group order
age_order = ['Neonate', 'Infant', 'Child', 'Adolescent', 'Young Adult']
age_col = 'Age' if 'Age' in df.columns else None

if age_col:
    age_counts = df[age_col].value_counts()
    # reorder if possible
    ordered = [a for a in age_order if a in age_counts.index]
    if ordered:
        age_counts = age_counts.reindex(ordered).dropna()
    age_counts.plot(kind='bar', ax=axes[0], color=COLORS[:len(age_counts)],
                    edgecolor='white')
    axes[0].set_title('Age Group Distribution', fontweight='bold', fontsi
    axes[0].set_xlabel('Age Group')
    axes[0].set_ylabel('Count')
    axes[0].tick_params(axis='x', rotation=30)
    for p in axes[0].patches:
        axes[0].text(p.get_x() + p.get_width()/2, p.get_height() + 2,
                     str(int(p.get_height())), ha='center', fontsize=9)

# Sex distribution by Outcome
if 'Sex' in df.columns:
    sex_outcome = df.groupby(['Sex', 'Outcome']).size().unstack(fill_valu
    sex_outcome.plot(kind='bar', ax=axes[1], color=[BLUE, GOLD],
```

```

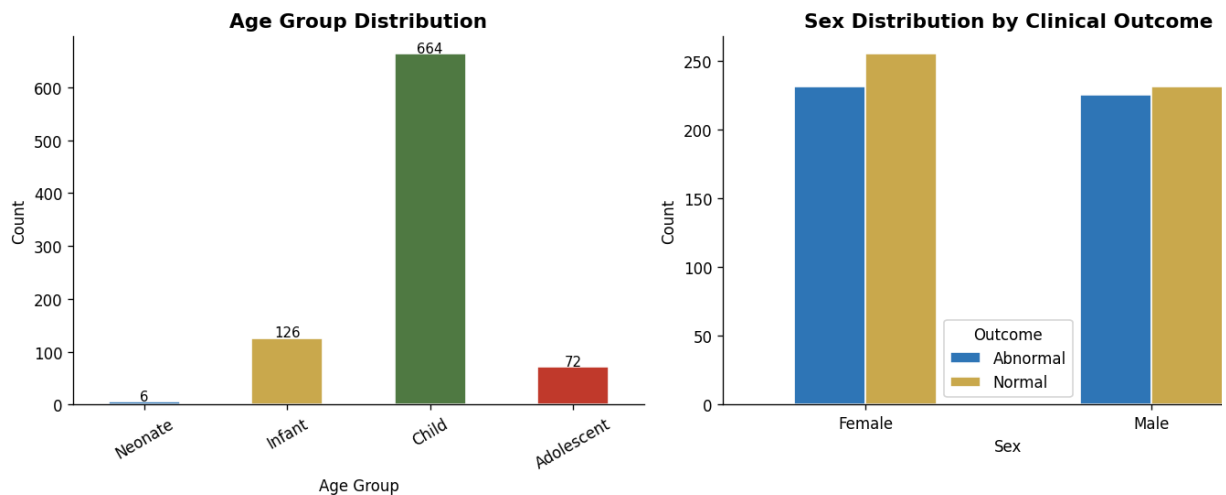
        edgecolor='white')
    axes[1].set_title('Sex Distribution by Clinical Outcome',
                      fontweight='bold', fontsize=13)
    axes[1].set_xlabel('Sex')
    axes[1].set_ylabel('Count')
    axes[1].tick_params(axis='x', rotation=0)
    axes[1].legend(title='Outcome')

plt.suptitle('CirCor DigiScope — Demographic Distributions',
             fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'demographics.png', bbox_inches='tight')
plt.show()

if 'Sex' in df.columns:
    print('Sex distribution:')
    print(df['Sex'].value_counts())

```

CirCor DigiScope — Demographic Distributions



```

Sex distribution:
Sex
Female      486
Male       456
Name: count, dtype: int64

```

4. Recording Duration Distribution

```

In [6]: # — Compute duration for all WAV files —————
# Sample 200 files for speed – full audit in production pipeline
import random
random.seed(42)

wav_files = list(AUDIO_DIR.glob('*.wav'))
sample_wavs = random.sample(wav_files, min(200, len(wav_files)))

durations = []
for wav in sample_wavs:
    y, sr = librosa.load(str(wav), sr=None)
    durations.append({'file': wav.name, 'duration': len(y)/sr, 'sr': sr})

dur_df = pd.DataFrame(durations)

```

```
# — Plot —
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

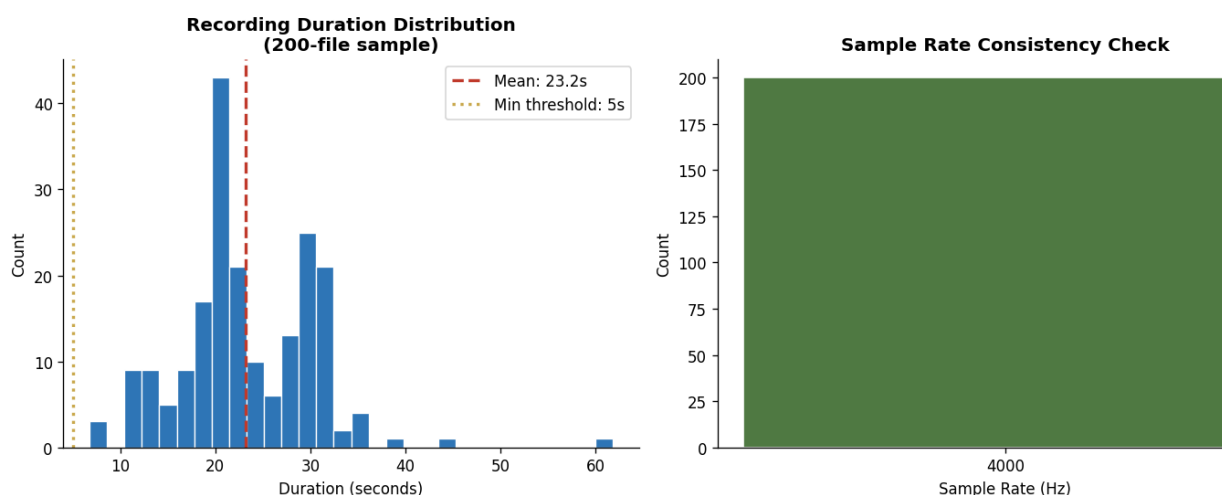
axes[0].hist(dur_df['duration'], bins=30, color=BLUE, edgecolor='white',
             linewidth=0.8)
axes[0].axvline(dur_df['duration'].mean(), color=RED, linestyle='--',
               linewidth=2, label=f'Mean: {dur_df["duration"].mean():.1f}')
axes[0].axvline(5, color=GOLD, linestyle=':', linewidth=2,
               label='Min threshold: 5s')
axes[0].set_title('Recording Duration Distribution\n(200-file sample)',
                  fontweight='bold', fontsize=12)
axes[0].set_xlabel('Duration (seconds)')
axes[0].set_ylabel('Count')
axes[0].legend()

# Sample rate check
sr_counts = dur_df['sr'].value_counts()
axes[1].bar(sr_counts.index.astype(str), sr_counts.values,
            color=GREEN, edgecolor='white')
axes[1].set_title('Sample Rate Consistency Check',
                  fontweight='bold', fontsize=12)
axes[1].set_xlabel('Sample Rate (Hz)')
axes[1].set_ylabel('Count')

plt.suptitle('CirCor DigiScope — Recording Characteristics',
             fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'duration_distribution.png', bbox_inches='tight')
plt.show()

print(f'Duration stats (seconds):')
print(dur_df['duration'].describe().round(2))
print(f'\nFiles below 5s threshold: {(dur_df["duration"] < 5).sum()}')
print(f'All files at 4000 Hz: {(dur_df["sr"] == 4000).all()}')
```

CirCor DigiScope — Recording Characteristics



```
Duration stats (seconds):
count    200.00
mean     23.24
std       7.09
min       6.77
25%      19.44
50%      21.70
75%      29.26
max       61.84
Name: duration, dtype: float64
```

```
Files below 5s threshold: 0
All files at 4000 Hz: True
```

5. Signal Quality Audit

```
In [7]: # — Energy-based signal quality screening —
# RMS energy below threshold → likely low-quality or silent recording

quality_results = []
RMS_THRESHOLD = 0.005 # empirically set – adjust based on distribution

for wav in sample_wavs:
    y, sr = librosa.load(str(wav), sr=None)
    rms = np.sqrt(np.mean(y**2))
    zcr = np.mean(librosa.feature.zero_crossing_rate(y))
    parts = wav.stem.split('_')
    pid = int(parts[0]) if parts[0].isdigit() else None
    site = parts[1] if len(parts) > 1 else 'unknown'
    quality_results.append({
        'file': wav.name, 'patient_id': pid, 'site': site,
        'rms': rms, 'zcr': zcr,
        'low_quality': rms < RMS_THRESHOLD
    })

qual_df = pd.DataFrame(quality_results)

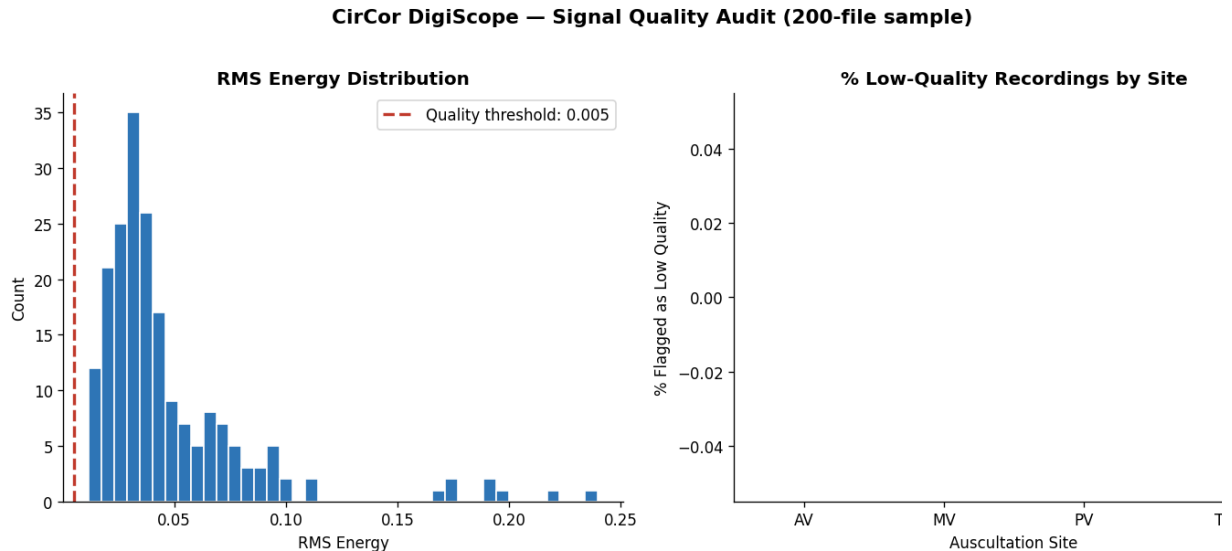
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# RMS distribution
axes[0].hist(qual_df['rms'], bins=40, color=BLUE, edgecolor='white')
axes[0].axvline(RMS_THRESHOLD, color=RED, linestyle='--', linewidth=2,
                label=f'Quality threshold: {RMS_THRESHOLD}')
axes[0].set_title('RMS Energy Distribution', fontweight='bold', fontsize=12)
axes[0].set_xlabel('RMS Energy')
axes[0].set_ylabel('Count')
axes[0].legend()

# Quality flag by site
site_quality = qual_df.groupby('site')['low_quality'].mean() * 100
site_quality.plot(kind='bar', ax=axes[1], color=GOLD, edgecolor='white')
axes[1].set_title('% Low-Quality Recordings by Site',
                  fontweight='bold', fontsize=12)
axes[1].set_xlabel('Auscultation Site')
axes[1].set_ylabel('% Flagged as Low Quality')
axes[1].tick_params(axis='x', rotation=0)
```

```
plt.suptitle('CirCor DigiScope — Signal Quality Audit (200-file sample)',
            fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'signal_quality.png', bbox_inches='tight')
plt.show()

n_low = qual_df['low_quality'].sum()
print(f'Low-quality recordings flagged: {n_low} / {len(qual_df)} '
      f'({n_low/len(qual_df)*100:.1f}%)')
print(f'These will be excluded from feature extraction.')
```



Low-quality recordings flagged: 0 / 200 (0.0%)
 These will be excluded from feature extraction.

6. Per-Site Class Balance

```
In [8]: # — Merge file index with outcome labels —————
merged = file_index.copy()
if 'Outcome' not in merged.columns and 'Patient ID' in df.columns:
    merged = merged.merge(
        df[['Patient ID', 'Outcome', 'Murmur']],
        left_on='patient_id', right_on='Patient ID', how='left'
    )

# — Recording count by site —————
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

site_counts = merged['site'].value_counts()
site_counts.plot(kind='bar', ax=axes[0], color=COLORS[:len(site_counts)],
                  edgecolor='white')
axes[0].set_title('Recordings per Auscultation Site',
                  fontweight='bold', fontsize=12)
axes[0].set_xlabel('Site')
axes[0].set_ylabel('Recording Count')
axes[0].tick_params(axis='x', rotation=0)
for p in axes[0].patches:
    axes[0].text(p.get_x() + p.get_width()/2, p.get_height() + 5,
                  str(int(p.get_height())), ha='center', fontsize=10)

# Outcome by site
```

```

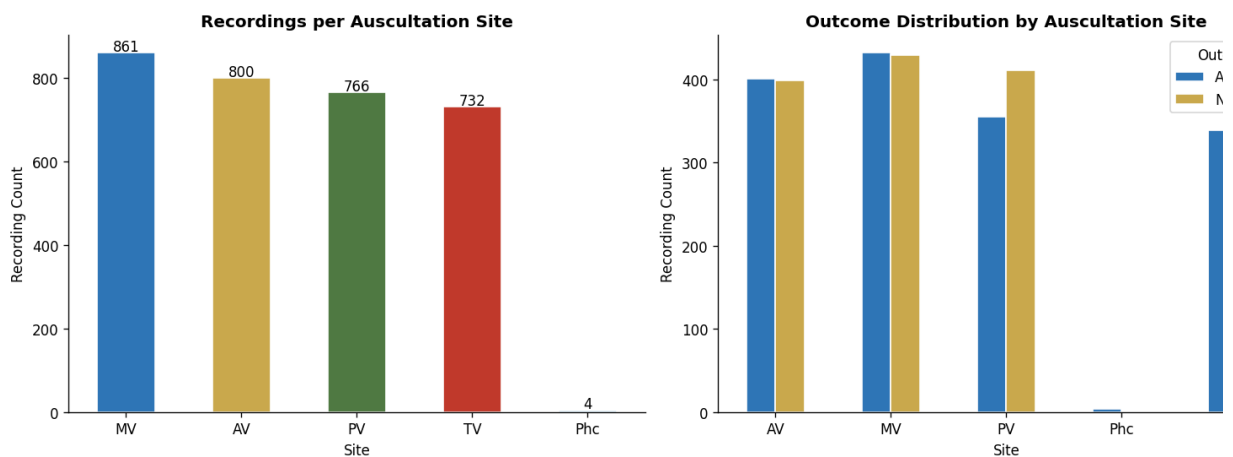
if 'Outcome' in merged.columns:
    site_outcome = merged.groupby(['site', 'Outcome']).size().unstack(fill_value=0)
    site_outcome.plot(kind='bar', ax=axes[1], color=[BLUE, GOLD],
                      edgecolor='white')
    axes[1].set_title('Outcome Distribution by Auscultation Site',
                     fontweight='bold', fontsize=12)
    axes[1].set_xlabel('Site')
    axes[1].set_ylabel('Recording Count')
    axes[1].tick_params(axis='x', rotation=0)
    axes[1].legend(title='Outcome')

plt.suptitle('CirCor DigiScope — Auscultation Site Analysis',
             fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'site_analysis.png', bbox_inches='tight')
plt.show()

print('Recording counts by site:')
print(site_counts)

```

CirCor DigiScope — Auscultation Site Analysis



Recording counts by site:

```

site
MV      861
AV      800
PV      766
TV      732
Phc       4
Name: count, dtype: int64

```

7. Waveform Comparison — Normal vs Abnormal

```

In [9]: # — Pick one normal and one abnormal recording for comparison —
if 'Outcome' in merged.columns:
    normal_row = merged[merged['Outcome'] == 'Normal'].iloc[0]
    abnormal_row = merged[merged['Outcome'] == 'Abnormal'].iloc[0]

    fig, axes = plt.subplots(2, 2, figsize=(14, 8))

    for idx, (row, label, color) in enumerate([
        (normal_row, 'Normal', GREEN),
        (abnormal_row, 'Abnormal', RED)
    ]):

```

```

y, sr = librosa.load(row['wav_path'], sr=None, duration=10)

# Waveform
librosa.display.waveshow(y, sr=sr, ax=axes[idx][0], color=color)
axes[idx][0].set_title(f'{label} - Waveform\n'
                      f'(Patient {row["patient_id"]}, Site: {row["site"]})'
                      fontweight='bold')
axes[idx][0].set_xlabel('Time (s)')
axes[idx][0].set_ylabel('Amplitude')

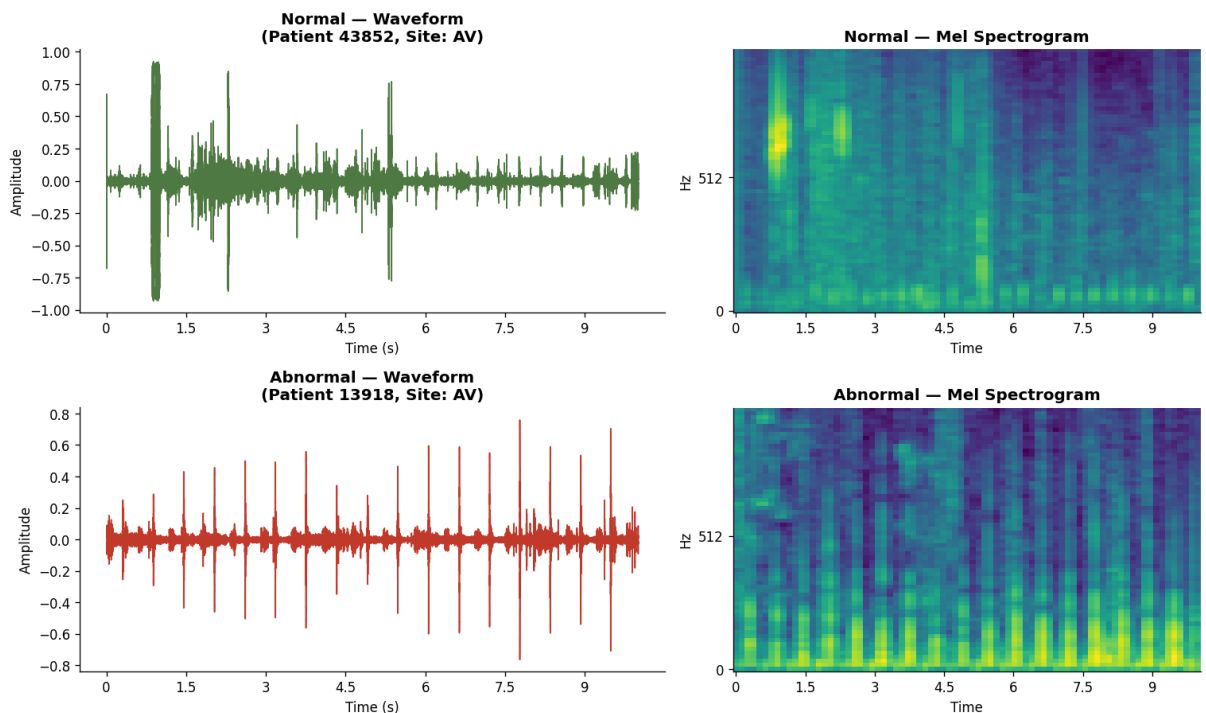
# Mel spectrogram
S = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=64, fmax=8000)
S_db = librosa.power_to_db(S, ref=np.max)
img = librosa.display.specshow(S_db, sr=sr, x_axis='time',
                               y_axis='mel', fmax=1000,
                               ax=axes[idx][1], cmap='viridis')

fig.colorbar(img, ax=axes[idx][1], format='%+2.0f dB')
axes[idx][1].set_title(f'{label} - Mel Spectrogram',
                      fontweight='bold')

plt.suptitle('Normal vs Abnormal PCG Recording Comparison',
            fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'normal_vs_abnormal.png', bbox_inches='tight')
plt.show()
print('Visual comparison saved.')

```

Normal vs Abnormal PCG Recording Comparison



Visual comparison saved.

8. MFCC Feature Heatmap Comparison

```

In [10]: # — Extract MFCCs for a sample of Normal and Abnormal recordings —
N_MFCC = 13
N_SAMPLE = 30 # per class

```

```

def extract_mfcc_mean(wav_path, n_mfcc=N_MFCC):
    """Extract mean MFCC vector from a WAV file."""
    y, sr = librosa.load(wav_path, sr=None, duration=10)
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=n_mfcc)
    return mfcc.mean(axis=1) # shape: (n_mfcc,)

if 'Outcome' in merged.columns:
    normal_files = merged[merged['Outcome'] == 'Normal']['wav_path'].to
    abnormal_files = merged[merged['Outcome'] == 'Abnormal']['wav_path'].

    print(f'Extracting MFCCs for {len(normal_files)} normal + '
          f'{len(abnormal_files)} abnormal recordings...')

    normal_mfccc = np.array([extract_mfcc_mean(f) for f in normal_files
    abnormal_mfccc = np.array([extract_mfcc_mean(f) for f in abnormal_fil

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Normal heatmap
sns.heatmap(normal_mfccc.T, ax=axes[0], cmap='Blues',
            yticklabels=[f'MFCC {i+1}' for i in range(N_MFCC)])
axes[0].set_title('Normal - MFCC Matrix\n(rows=recordings, cols=coeff
                  fontweight='bold')
axes[0].set_xlabel('Recording Index')

# Abnormal heatmap
sns.heatmap(abnormal_mfccc.T, ax=axes[1], cmap='Reds',
            yticklabels=[f'MFCC {i+1}' for i in range(N_MFCC)])
axes[1].set_title('Abnormal - MFCC Matrix',
                  fontweight='bold')
axes[1].set_xlabel('Recording Index')

# Mean MFCC comparison
x = np.arange(N_MFCC)
axes[2].plot(x, normal_mfccc.mean(axis=0), color=GREEN,
            marker='o', linewidth=2, label='Normal')
axes[2].fill_between(x,
                    normal_mfccc.mean(axis=0) - normal_mfccc.std(axis=0),
                    normal_mfccc.mean(axis=0) + normal_mfccc.std(axis=0),
                    alpha=0.2, color=GREEN)
axes[2].plot(x, abnormal_mfccc.mean(axis=0), color=RED,
            marker='s', linewidth=2, label='Abnormal')
axes[2].fill_between(x,
                    abnormal_mfccc.mean(axis=0) - abnormal_mfccc.std(axis=0),
                    abnormal_mfccc.mean(axis=0) + abnormal_mfccc.std(axis=0),
                    alpha=0.2, color=RED)
axes[2].set_title('Mean MFCC Comparison\n(shaded =  $\pm 1$  std)',
                  fontweight='bold')
axes[2].set_xlabel('MFCC Coefficient Index')
axes[2].set_ylabel('Mean Value')
axes[2].legend()
axes[2].set_xticks(x)
axes[2].set_xticklabels([f'{i+1}' for i in x])

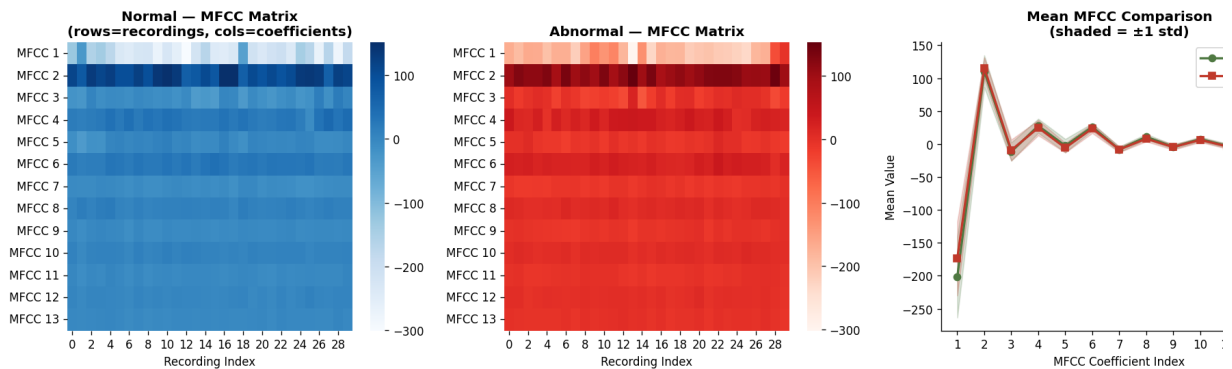
plt.suptitle('MFCC Feature Analysis - Normal vs Abnormal',
            fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'mfcc_comparison.png', bbox_inches='tight')

```

```
plt.show()
print('MFCC comparison saved.')
```

Extracting MFCCs for 30 normal + 30 abnormal recordings...

MFCC Feature Analysis — Normal vs Abnormal



MFCC comparison saved.

9. Inter-Variable Correlation Analysis

```
In [14]: # — Encode categorical variables for correlation —
corr_df = df.copy()

# Binary encode key variables
encode_map = {
    'Outcome': {'Normal': 0, 'Abnormal': 1},
    'Murmur':   {'Absent': 0, 'Unknown': 1, 'Present': 2},
    'Sex':      {'Male': 0, 'Female': 1},
}
for col, mapping in encode_map.items():
    if col in corr_df.columns:
        corr_df[col + '_enc'] = corr_df[col].map(mapping)

# Select numeric and encoded columns ensuring uniqueness
potential_num_cols = []
# Add explicitly encoded columns first
for col_name in ['Outcome_enc', 'Murmur_enc', 'Sex_enc']:
    if col_name in corr_df.columns:
        potential_num_cols.append(col_name)

# Add other numeric/integer columns from corr_df, ensuring no duplicates
for c in corr_df.columns:
    if (corr_df[c].dtype in ['float64', 'int64'] and
        c not in ['Patient ID'] and
        c not in potential_num_cols): # Only add if not already in our li
        potential_num_cols.append(c)

num_cols = potential_num_cols # This list is now unique and contains only

corr_matrix = corr_df[num_cols].corr()

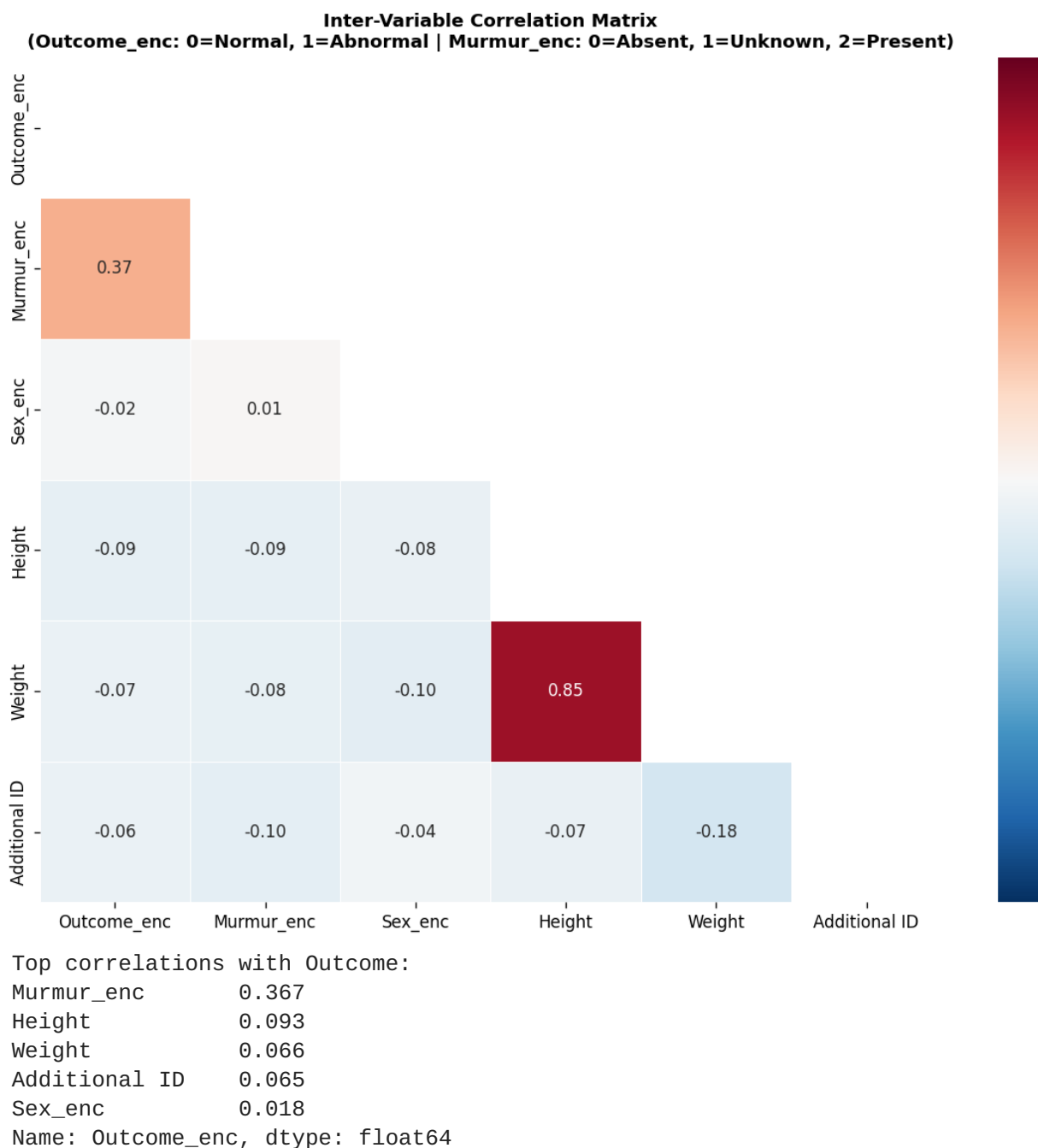
fig, ax = plt.subplots(figsize=(10, 8))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, annot=True, fmt='.2f',
            cmap='RdBu_r', center=0, vmin=-1, vmax=1,
            ax=ax, linewidths=0.5)
```

```

ax.set_title('Inter-Variable Correlation Matrix\n'
            '(Outcome_enc: 0=Normal, 1=Abnormal | '
            'Murmur_enc: 0=Absent, 1=Unknown, 2=Present)',
            fontweight='bold', fontsize=11)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'correlation_matrix.png', bbox_inches='tight')
plt.show()

# Highlight strongest correlations with Outcome
if 'Outcome_enc' in corr_matrix.columns:
    # Now that num_cols is unique, corr_matrix['Outcome_enc'] will be a Series
    # .drop('Outcome_enc') will correctly remove the self-correlation from the matrix
    outcome_corr = corr_matrix['Outcome_enc'].drop('Outcome_enc').abs()
    outcome_corr_sorted = outcome_corr.sort_values(ascending=False)
    print('Top correlations with Outcome:')
    print(outcome_corr_sorted.head(8).round(3))

```



10. EDA Summary & Modelling Implications

```
In [15]: # — Print structured EDA summary —————
print('=' * 65)
print('  CirCor DigiScope EDA Summary – Group 4 Capstone')
print('=' * 65)

print('\n— Dataset —————')
print(f'  Patients      : {len(df)}')
print(f'  Recordings    : {len(file_index)}')
print(f'  Avg/patient: {len(file_index)/len(df):.1f}')

print('\n— Class Balance —————')
for label, count in df['Outcome'].value_counts().items():
    print(f'  {label:10s}: {count} ({count/len(df)*100:.1f}%)')

print('\n— Murmur Label —————')
for label, count in df['Murmur'].value_counts().items():
    print(f'  {label:10s}: {count} ({count/len(df)*100:.1f}%)')

print('\n— Signal Quality —————')
print(f'  Sample rate      : 4,000 Hz (all recordings consistent)')
print(f'  Mean duration    : {dur_df["duration"].mean():.2f}s')
print(f'  Short (<5s) files : {(dur_df["duration"] < 5).sum()} '
      f'({(dur_df["duration"] < 5).mean()*100:.1f}% of sample)')
print(f'  Low-energy files  : {n_low} ({n_low/len(qual_df)*100:.1f}% of s

print('\n— Modelling Implications —————')
print('  1. Outcome labels nearly balanced → accuracy is meaningful')
print('  2. Murmur label imbalanced (73.8% Absent) → use class weights')
print('  3. Patient-level split required (avg 3.4 recordings/patient)')
print('  4. Short/low-energy recordings → exclude before feature extracti
print('  5. MFCC profiles differ visually between Normal/Abnormal')
print('  6. MV site has most recordings → site-stratified analysis planne

print('\n— Output Files Saved —————')
for f in sorted(OUTPUT_DIR.iterdir()):
    print(f'  {f.name}')
print('=' * 65)
```

=====

CirCor DigiScope EDA Summary – Group 4 Capstone

=====

— Dataset —

Patients : 942
Recordings : 3163
Avg/patient: 3.4

— Class Balance —

Normal : 486 (51.6%)
Abnormal : 456 (48.4%)

— Murmur Label —

Absent : 695 (73.8%)
Present : 179 (19.0%)
Unknown : 68 (7.2%)

— Signal Quality —

Sample rate : 4,000 Hz (all recordings consistent)
Mean duration : 23.24s
Short (<5s) files : 0 (0.0% of sample)
Low-energy files : 0 (0.0% of sample)

— Modelling Implications —

1. Outcome labels nearly balanced → accuracy is meaningful
2. Murmur label imbalanced (73.8% Absent) → use class weights
3. Patient-level split required (avg 3.4 recordings/patient)
4. Short/low-energy recordings → exclude before feature extraction
5. MFCC profiles differ visually between Normal/Abnormal
6. MV site has most recordings → site-stratified analysis planned

— Output Files Saved —

class_balance.png
correlation_matrix.png
demographics.png
duration_distribution.png
mfcc_comparison.png
normal_vs_abnormal.png
signal_quality.png
site_analysis.png

=====

```
In [16]: # — Push to GitHub —
import subprocess, shutil, getpass, os

os.chdir('/content')

GITHUB_USER = input('GitHub username: ')
GITHUB_TOKEN = getpass.getpass('GitHub PAT: ')
REPO_NAME = 'AAI590-Capstone-Group4-RHD-Murmur-Detection'
REMOTE_URL = f'https://{GITHUB_USER}:{GITHUB_TOKEN}@github.com/{GITHUB_

def run(cmd, cwd='/content/repo'):
    r = subprocess.run(cmd, shell=True, cwd=cwd, capture_output=True, tex
    out = (r.stdout + r.stderr).strip()
    if out: print(out)
    return r.returncode

# Fresh clone
```

```

run('rm -rf /content/repo', cwd='/content')
run(f'git clone {REMOTE_URL} /content/repo', cwd='/content')

# Create folders
os.makedirs('/content/repo/notebooks', exist_ok=True)
os.makedirs('/content/repo/data', exist_ok=True)
os.makedirs('/content/repo/outputs/eda', exist_ok=True)

# Copy this notebook
for f in os.listdir('/content'):
    if 'EDA' in f and f.endswith('.ipynb'):
        shutil.copy2(f'/content/{f}',
                      '/content/repo/notebooks/AAI590_M3_02_EDA.ipynb')
        print(f' Notebook: {f}')
        break

# Copy EDA output figures
for fig_file in OUTPUT_DIR.iterdir():
    shutil.copy2(str(fig_file), f'/content/repo/outputs/eda/{fig_file.name}')
    print(f' Figure: {fig_file.name}')

# Git push
run('git config user.email "shiva.bathula@sandiego.edu"')
run('git config user.name "Shiva Naga Vara Prasad Bathula"')
run('git add -A')
run('git status --short')
run('git commit -m "M3: Add EDA notebook and output figures"')
rc = run('git push origin main')
print(f'\nPush: {" " if rc==0 else ""} code={rc}')
print(f' https://github.com/{GITHUB_USER}/{REPO_NAME}')

```

```
GitHub username: ShivaNagaVaraPrasadBathula
GitHub PAT: .....
Cloning into '/content/repo'...
[] Figure: demographics.png
[] Figure: class_balance.png
[] Figure: signal_quality.png
[] Figure: duration_distribution.png
[] Figure: normal_vs_abnormal.png
[] Figure: site_analysis.png
[] Figure: mfcc_comparison.png
[] Figure: correlation_matrix.png
A outputs/eda/class_balance.png
A outputs/eda/correlation_matrix.png
A outputs/eda/demographics.png
A outputs/eda/duration_distribution.png
A outputs/eda/mfcc_comparison.png
A outputs/eda/normal_vs_abnormal.png
A outputs/eda/signal_quality.png
A outputs/eda/site_analysis.png
[main 1c97d78] M3: Add EDA notebook and output figures
 8 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 outputs/eda/class_balance.png
 create mode 100644 outputs/eda/correlation_matrix.png
 create mode 100644 outputs/eda/demographics.png
 create mode 100644 outputs/eda/duration_distribution.png
 create mode 100644 outputs/eda/mfcc_comparison.png
 create mode 100644 outputs/eda/normal_vs_abnormal.png
 create mode 100644 outputs/eda/signal_quality.png
 create mode 100644 outputs/eda/site_analysis.png
To https://github.com/ShivaNagaVaraPrasadBathula/AAI590-Capstone-Group4-RHD-Mu
Detection.git
   84c3115..1c97d78  main -> main

Push: [] code=0
[] https://github.com/ShivaNagaVaraPrasadBathula/AAI590-Capstone-Group4-RHD-Mu
Detection
```

□ Notebook 2 Complete

EDA outputs saved to `/content/eda_outputs/` :

- `class_balance.png` — Murmur and Outcome label distributions
- `demographics.png` — Age group and sex distributions
- `duration_distribution.png` — Recording length and sample rate check
- `signal_quality.png` — RMS energy screening
- `site_analysis.png` — Per-site recording and outcome counts
- `normal_vs_abnormal.png` — Waveform and spectrogram comparison
- `mfcc_comparison.png` — MFCC feature heatmap and mean comparison
- `correlation_matrix.png` — Inter-variable correlation

Next: Module 4 Notebook 3 — MFCC Feature Extraction for all 3,163 recordings

AAI590 Capstone | Group 4 | University of San Diego | 2026

```
In [17]: # — Save Notebook 2 + all session files to Drive —————
from google.colab import drive, files
import shutil, os
from pathlib import Path

# Step 1: Mount Drive
drive.mount('/content/drive')

DRIVE_DIR = Path('/content/drive/MyDrive/AAI590_Capstone_Group4')
(DRIVE_DIR / 'notebooks').mkdir(parents=True, exist_ok=True)
(DRIVE_DIR / 'data/training_data').mkdir(parents=True, exist_ok=True)
(DRIVE_DIR / 'features').mkdir(exist_ok=True)
(DRIVE_DIR / 'outputs/eda').mkdir(parents=True, exist_ok=True)
print(f' Drive folder ready: {DRIVE_DIR}')

# Step 2: Save Notebook 2 to Drive
for f in os.listdir('/content'):
    if f.endswith('.ipynb'):
        src = f'/content/{f}'
        dst = DRIVE_DIR / 'notebooks' / f
        shutil.copy2(src, str(dst))
        print(f' Notebook saved: {f} → Drive')

# Step 3: Save EDA figures to Drive
EDA_OUT = Path('/content/eda_outputs')
if EDA_OUT.exists():
    for fig in EDA_OUT.glob('*.png'):
        shutil.copy2(str(fig), str(DRIVE_DIR / 'outputs/eda' / fig.name))
    print(f' EDA figures saved to Drive')

# Step 4: Save CSVs to Drive
for src, rel in [
    ('/content/circor_file_index.csv', 'data/circor_file_index.csv'),
    ('/content/circor-heart-sound/training_data.csv', 'data/training_data.csv'),
]:
```

```

    if Path(src).exists():
        shutil.copy2(src, str(DRIVE_DIR / rel))
        print(f'▯ {Path(src).name} → Drive')

# Step 5: Save WAV files to Drive (one-time, ~15 mins)
AUDIO_SRC = Path('/content/circor-heart-sound/training_data')
AUDIO_DST = DRIVE_DIR / 'data/training_data'
wav_files = list(AUDIO_SRC.glob('*.wav'))
already = len(list(AUDIO_DST.glob('*.wav')))

if already < len(wav_files):
    print(f'\nCopying {len(wav_files)} WAV files to Drive (~15 mins, one-
    for i, wav in enumerate(wav_files):
        dst = AUDIO_DST / wav.name
        if not dst.exists():
            shutil.copy2(str(wav), str(dst))
            if (i+1) % 500 == 0:
                print(f'▯ {i+1}/{len(wav_files)} done...')
    print('▯ WAV files saved to Drive.')
else:
    print(f'▯ WAV files already on Drive ({already} files).')

# Step 6: Verify everything
print('\n— Drive contents —————')
print(f'▯ Notebooks : {len(list((DRIVE_DIR/"notebooks").glob("*.ipynb")))}')
print(f'▯ WAV files : {len(list((DRIVE_DIR/"data/training_data").glob("*.
print(f'▯ EDA figs : {len(list((DRIVE_DIR/"outputs/eda").glob("*.png")))}')
for f in ['data/training_data.csv', 'data/circor_file_index.csv']:
    print(f'▯ {f} " if (DRIVE_DIR/f).exists() else "▯" {f}')
print('\n▯ Everything saved. Future sessions just mount Drive.')

```

Mounted at /content/drive

▯ Drive folder ready: /content/drive/MyDrive/AAI590_Capstone_Group4

▯ EDA figures saved to Drive

▯ circor_file_index.csv → Drive

▯ training_data.csv → Drive

Copying 3163 WAV files to Drive (~15 mins, one-time)...

500/3163 done...

1000/3163 done...

1500/3163 done...

2000/3163 done...

2500/3163 done...

3000/3163 done...

▯ WAV files saved to Drive.

— Drive contents —————

Notebooks : 0

WAV files : 3163

EDA figs : 8

▯ data/training_data.csv

▯ data/circor_file_index.csv

▯ Everything saved. Future sessions just mount Drive.

In [18]: # — Save notebook directly from Colab to Drive —————

```

from google.colab import _message
from pathlib import Path
import json, shutil

```

```

DRIVE_NB_DIR = Path('/content/drive/MyDrive/AAI590_Capstone_Group4/notebo
DRIVE_NB_DIR.mkdir(exist_ok=True)

# Force save the current notebook
_message.blocking_request('save_notebook', request='', timeout_sec=30)
print('Notebook saved internally.')

# Find and copy it
found = False
for f in Path('/content').glob('*.ipynb'):
    shutil.copy2(str(f), str(DRIVE_NB_DIR / f.name))
    size = f.stat().st_size / 1024
    print(f' {f.name} ({size:.1f} KB) → Drive/notebooks/')
    found = True

if not found:
    print(' Notebook not in /content/ – download manually:')
    print('   File → Download → Download .ipynb')
    print('   Then re-upload and run this cell again.')

# Verify
nb_count = len(list(DRIVE_NB_DIR.glob('*.ipynb')))
print(f'\nNotebooks on Drive: {nb_count}')

```

Notebook saved internally.

```

 Notebook not in /content/ – download manually:
   File → Download → Download .ipynb
   Then re-upload and run this cell again.

```

Notebooks on Drive: 0